

C05-AT1

Pseudocode writing

Name: Manoj M

Reg no: 192511060

course name: data structures

course code: CSA0306

1. Topologies sort with higher - priority tie-breaking

Algo priority (V, E, P)

FOR EACH node in V DO

inDegree [u] $\leftarrow$ 0

END FOR

FOR EACH edge (u, v) in E DO

inDegree [v] $\leftarrow$ inDegree [v] + 1

IF inDegree [u] == 0 THEN

PA.insert (u)

END IF

END FOR

2. Dijkstra with avoided nodes constraint.

Algo constrain (V, E, m, st, A)

IF S IN A OR IN A THEN

RETURN "NO valid path"

END IF

FOR EACH vertex x in V DO

dist [v] $\leftarrow$ infinity

prev[v] $\leftarrow$ NULL

END FOR

4.

```

IF altcost < dist[v] THEN
    dist[v] ← altcost
    prev[v] ← u
    Q. insert (v, dist[v])
END IF

```

```

END FOR

```

3. Network Routing validation.

Algo validate (v, E)

isvalid ← true

FOR EACH edge (u, v) in E DO

IF u == v THEN

PRINT "violation"

END IF

END FOR

visit count ← 0

Initialize queue Q

Initialize set visited

start node ← v[0]

Q. Enqueue (start node)

visited. add (start node)

END FOR

END ALGO

4. shortest path via specific intermediate node

Algo shortest path (V, E, S, W, K, D)

$(dist_1, path_1) \leftarrow \text{standard Dijkstra}(V, E, W, S, K)$

$(dist_2, path_2) \leftarrow \text{standard Dijkstra}(V, E, W, S, K)$

IF $dist_1 == \text{infinity}$ OR $dist_2 == \text{infinity}$ THEN

RETURN "NO path exists"

END IF

Full path $\leftarrow \text{combine paths}(path_1, path_2)$

END Algo.

5. early - stopping BFS for social networks

Algo BFS early (V, E, S, T)

IF $S == T$ THEN

RETURN 0

END IF

WHILE Q is NOT EMPTY DO

curr $\leftarrow Q.dequeue()$

FOR each neighbor V of curr DO

IF V NOT in visited DO

visited.add(V)

IF $V == T$ THEN

RETURN distance $[V]$

END IF.

RETURN -1

6) Maximum spanning tree via modified Kruskal's

Algo MaxSpan(V, E, R)

Sort edge E by weight $R(e)$ descending

FOR each edge $e = (u, v)$ in E DO

IF DS-Find(u) $\neq$ DS-Find(v) THEN

Max. around(e)

DS-Union(u, v)

END IF

END FOR

END Algo.

7. Dijkstra with node congestion penalties

Algo DijkstraNode(V, E, w, P, S, T)

FOR EACH vertex v in $V \setminus S$ DO

dist[v] $\leftarrow$ infinity

END FOR

dist[S] $\leftarrow P[S]$

PQ-insert(S , dist[S])

WHILE PQ is not empty DO

$u \leftarrow$ PQ-extractMin()

IF $u = T$ THEN

RETURN dist[T]

END IF

END WHILE

END Algo

42/7 multi-source BFS for emergency response:

algort multisource (V, E, s, T)

FOR EACH station in source DO

IF station == T THEN

RETURN station, 0

END IF

Q.enqueue (station)

visited.add (station)

original station [station] $\leftarrow$ station

dist [station] $\leftarrow$ 0

END FOR

WHILE Q IS NOT EMPTY DO

cur $\leftarrow$ Q.dequeue ()

FOR EACH neighbor in visited FOR

visited.add (v)

dist [v] $\leftarrow$ dist [cur] + 1

original station [v] $\leftarrow$ original [cur]

IF v == T THEN

RETURN original [v], dist [v]

END IF

END WHILE

END Algort

9. Lexicographical topological sort

Algo lexico (V, E)

FOR EACH node u in V DO

indegree [u] $\leftarrow 0$

END FOR

FOR EACH edge (u, v) in E DO

indegree [v] $\leftarrow$ indegree [v] + 1

END FOR

FOR EACH node v from 1 to n

IF indegree [v] == 0 THEN

QA. insert (v)

ENDIF

END FOR

10. Maximum bottleneck capacity path

Algo max path (v, E, c, s, d)

FOR EACH vertex v in V DO

maxcap [v] $\leftarrow 0$

path [v] $\leftarrow$ NULL

maxcap [s] $\leftarrow$ infinity

WHILE $u \neq d$ THEN

BREAK

END WHILE

FOR EACH neighbor u of v DO

bottleneck $\leftarrow$ minpath (c, u, v)

END FOR

END WHILE

END Algo